

# Pipeline CI/CD — Rock in Rio Backend

Documentação operacional completa de build, teste, segurança e deploy da API Achados & Perdidos (Spring Boot + Docker + GitLab CI).

|                    |                                                                        |
|--------------------|------------------------------------------------------------------------|
| Aplicação          | api-achados-perdidos — Spring Boot 4.0.2 / Java 25                     |
| Orquestração CI/CD | GitLab CI/CD — runner com tag deploy                                   |
| Ambientes          | develop → Development · main → Production                              |
| Público-alvo       | Equipe DevOps / SRE responsável pela execução e manutenção do pipeline |
| Emissão            | 24/07/2026 — Versão 1.1                                                |

**Revisão 1.1.** Acrescenta: seção consolidada de dependências externas (§2.4) e o **health check agora implementado** no container (Dockerfile + compose + Actuator restrito a /health) — §10.3 e §15.

# Sumário

|    |                                                    |
|----|----------------------------------------------------|
| 01 | Sumário executivo e visão geral                    |
| 02 | Stack, topologia e dependências externas           |
| 03 | Pré-requisitos de infraestrutura (runner e VPS)    |
| 04 | Variáveis de ambiente e gestão de segredos         |
| 05 | Anatomia do pipeline — <code>.gitlab-ci.yml</code> |
| 06 | Stage build — imagem e extração do fat-jar         |
| 07 | Stage test — MySQL efêmero e gate real             |
| 08 | Stage security — SCA, SAST e secret scanning       |
| 09 | Stage deploy — publicação em dev e produção        |
| 10 | Dockerfile multi-stage, runtime e health check     |
| 11 | Nginx, Cloudflare e exposição da API               |
| 12 | Banco de dados e migrations SQL                    |
| 13 | Procedimento de bootstrap (primeiro deploy)        |
| 14 | Deploy contínuo, rollback e limpeza                |
| 15 | Observabilidade, health check e troubleshooting    |
| 16 | Checklist operacional e hardening                  |

## 1. Sumário executivo e visão geral

Este runbook descreve, de ponta a ponta, como o backend da aplicação Achados & Perdidos (Rock in Rio) é construído, testado, auditado e implantado automaticamente pelo GitLab CI/CD. O objetivo é permitir que qualquer pessoa da equipe DevOps execute, diagnostique e evolua o pipeline sem depender de conhecimento tácito.

- **4 estágios sequenciais:** build → test → security → deploy.
- **2 ambientes por branch:** `develop` (dev) e `main` (produção).
- **3 scanners de segurança:** Trivy (SCA), Semgrep (SAST), Gitleaks (segredos).
- **1 Dockerfile multi-stage:** base → test → build → runtime.

### 1.1 Fluxo macro do pipeline

| git push                             | BUILD                                  | TEST                                     | SECURITY                                      | DEPLOY                           |
|--------------------------------------|----------------------------------------|------------------------------------------|-----------------------------------------------|----------------------------------|
| develop / main<br>dispara o pipeline | build + extrai<br><code>app.jar</code> | MySQL efêmero +<br><code>mvn test</code> | SCA/SAST/Secrets — paralelo,<br>não bloqueia* | rsync + compose<br>(dev ou prod) |

\* Os jobs de segurança rodam com `allow_failure: true` — são visíveis mas não bloqueiam o deploy até que a equipe faça a primeira triagem de findings (ver §8).

### 1.2 Princípios de design adotados

- **Gate de teste real:** os testes de integração dependem de um MySQL de verdade. Sem banco, a suíte é pulada ( `assumeTrue` ) e o gate não validaria nada — por isso o CI sobe um MySQL efêmero antes de testar.
- **Build reproduzível e offline no SCA:** o Trivy escaneia o fat-jar já construído, sem re-resolver o Maven Central (evita rate-limit HTTP 429).
- **Deploy imutável por commit:** cada imagem é etiquetada com `ci-${CI_COMMIT_SHORT_SHA}` , permitindo rastreabilidade e rollback.
- **Segurança por camadas (OWASP):** SCA (A06), SAST (A05/A08), secret scanning, além de rate-limiting, JWT com fail-fast e CORS restritivo na aplicação.

- **Isolamento por ambiente:** `resource_group` serializa deploys concorrentes por ambiente, evitando corrida entre pipelines.

## 2. Stack, topologia e dependências externas

### 2.1 Stack da aplicação

| Camada                   | Tecnologia                           | Observações                                                                                    |
|--------------------------|--------------------------------------|------------------------------------------------------------------------------------------------|
| Linguagem / Runtime      | Java 25                              | Compilado e executado sobre Eclipse Temurin 25                                                 |
| Framework                | Spring Boot 4.0.2                    | Web MVC, Data JPA, Security, Validation, Mail                                                  |
| Segurança                | Spring Security + JWT (jjwt 0.12.6)  | IDs opacos assinados, Bucket4j para rate-limit                                                 |
| Persistência             | MySQL 8 · Hibernate                  | Banco <code>achados_perdidos</code> ; <code>ddl-auto=validate</code> em produção               |
| Armazenamento            | Disco local ou AWS S3                | SDK AWS 2.31.78; provider configurável em <code>/configuracoes</code>                          |
| Documentação             | springdoc-openapi 3.0.2 (Swagger UI) | Desligável via <code>SPRINGDOC_ENABLED=false</code> em prod                                    |
| Health / observabilidade | Spring Boot Actuator                 | <b>rev. 1.1</b> Somente <code>/actuator/health</code> exposto (restrito, sem detalhes) — §10.3 |
| Build                    | Maven 3.9                            | <code>finalName=app</code> → artefato <code>target/app.jar</code>                              |

### 2.2 Topologia de produção



**Domínio com underscore.** `api_rockinrio.xpto.com.br` contém `_`, que o Tomcat rejeita no header `Host` (HTTP 400). O Nginx envia ao upstream um `Host` válido com hífen (`api-rockinrio...`) e preserva o domínio real em `X-Forwarded-Host`. `X-Forwarded-Proto: https` informa à aplicação que o TLS foi terminado no Cloudflare.

### 2.3 Diretórios-chave no repositório

| Caminho                     | Papel no pipeline                                    |
|-----------------------------|------------------------------------------------------|
| <code>.gitlab-ci.yml</code> | Definição completa do pipeline (stages, jobs, rules) |

| Caminho                        | Papel no pipeline                                          |
|--------------------------------|------------------------------------------------------------|
| Dockerfile                     | Build multi-stage: base → test → build → runtime           |
| docker-compose.yml             | Runtime do container no servidor de deploy                 |
| .dockerignore / .gitleaks.toml | Contexto de build enxuto / allowlist de secret scanning    |
| scripts/*.sql                  | Migrations versionadas (018 ... 056) aplicadas ao MySQL    |
| scripts/ci-docker-cleanup.sh   | Limpeza pós-deploy (imagens/cache antigos)                 |
| deploy/nginx/*.conf            | Vhost do reverse proxy (referência de configuração)        |
| .env (no servidor)             | Segredos e configuração de runtime — <b>NÃO</b> versionado |

## 2.4 Dependências externas rev. 1.1

Consolidação das dependências externas — de runtime (produção) e de CI (rede de saída do runner). "Obrigatória" bloqueia a operação/pipeline se ausente.

| Dependência                    | Tipo    | Criticidade               | Como é consumida                                                                                                                                                                                                         |
|--------------------------------|---------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MySQL 8                        | Runtime | <b>Obrigatória</b>        | Banco <code>achados_perdidos</code> na VPS (3306), alcançado via <code>host.docker.internal</code> . Sem ele a app falha na validação de schema.                                                                         |
| Cloudflare                     | Runtime | <b>Obrigatória (prod)</b> | Edge HTTPS/TLS e WAF. X-Forwarded-Proto: <code>https</code> informa TLS terminado na borda.                                                                                                                              |
| Nginx (VPS)                    | Runtime | <b>Obrigatória</b>        | Reverse proxy → <code>127.0.0.1:8203</code> ; ajusta o header <code>Host</code> (underscore) — §11.                                                                                                                      |
| Servidor SMTP                  | Runtime | <b>Condicional</b>        | E-mail transacional do workflow de claims (Spring Mail). <b>Configurado em <code>/configuracoes</code> (persistido no banco), não por variável de ambiente.</b> Necessário apenas quando o envio de e-mail é habilitado. |
| AWS S3 (ou MinIO)              | Runtime | <b>Opcional</b>           | Provider de armazenamento de anexos, selecionável em <code>/configuracoes</code> . Credenciais via <code>AWS_*</code> ou IAM role — nunca no banco.                                                                      |
| Registro Docker / imagens base | CI      | <b>Obrigatória</b>        | <code>maven:3.9-eclipse-temurin-25</code> , <code>eclipse-temurin:25-jre-alpine</code> , <code>mysql:8.0</code> , <code>docker:24-cli</code> .                                                                           |
| Maven Central                  | CI      | <b>Obrigatória</b>        | <code>mvn dependency:go-offline</code> no estágio <code>base</code> do Dockerfile.                                                                                                                                       |
| Trivy DB · Semgrep · Gitleaks  | CI      | <b>Recomendada</b>        | Imagens <code>aquasec/trivy</code> , <code>semgrep/semgrep</code> (regras <code>p/java,p/secrets,p/owasp-top-ten</code> ), <code>zricethezav/gitleaks</code> . Jobs <code>allow_failure</code> — não bloqueiam.          |

O runner precisa de acesso de saída à internet para baixar essas imagens/bases (§3.1). Recomenda-se fixar digests das imagens `:latest` dos scanners para builds reprodutíveis (§16.2).

## 3. Pré-requisitos de infraestrutura

### 3.1 GitLab Runner

Todos os jobs declaram `tags: [deploy]`. É obrigatório existir ao menos um runner registrado com essa tag, com acesso ao Docker daemon.

- **Executor:** Shell ou Docker com socket montado — o pipeline invoca `docker build`, `docker run`, `docker compose` diretamente.
- **Imagem default dos jobs:** `docker:24-cli` com `pull_policy: if-not-present`.
- Docker Engine + plugin Compose v2 (o pipeline usa `docker compose`, não `docker-compose`).
- **Localização:** o mesmo runner que testa também faz o deploy — precisa enxergar `/opt/rockinrio-backend` na VPS e o MySQL de produção.
- **Rede:** acesso de saída à internet (Trivy DB, Semgrep, Gitleaks, MySQL, Temurin, Maven Central).

**Por que o runner é local à VPS.** O template de deploy faz `rsync` do repositório para `/opt/rockinrio-backend` e sobe o compose na própria máquina do runner. Ou seja, o runner com a tag `deploy` deve rodar no servidor de destino (ou ter diretório/daemon compartilhados). O stage `test` usa `--network host`, pressupondo runner Linux com host networking.

### 3.2 Servidor de deploy (VPS)

| Requisito           | Detalhe                                                                                                          |
|---------------------|------------------------------------------------------------------------------------------------------------------|
| Sistema             | Linux com Docker Engine 24+ e Compose v2                                                                         |
| Diretório de deploy | <code>/opt/rockinrio-backend</code> (variável <code>DEPLOY_DIR</code> )                                          |
| Arquivo de segredos | <code>/opt/rockinrio-backend/.env</code> — criado manualmente antes do 1º deploy                                 |
| MySQL               | Instância acessível via <code>host.docker.internal</code> (mapeado por <code>host-gateway</code> ) na porta 3306 |
| Nginx               | Reverse proxy para <code>127.0.0.1:8203</code> (produção) — \$11                                                 |
| Portas              | 3306 ocupada pelo banco; o container publica em <code>HOST_PORT</code> (dev 8080, prod 8203)                     |
| Ferramentas         | <code>rsync</code> (instalado automaticamente pelo <code>before_script</code> se ausente)                        |

## 4. Variáveis de ambiente e gestão de segredos

### 4.1 Variáveis definidas no pipeline

| Variável                          | Valor                                                     | Papel                                                            |
|-----------------------------------|-----------------------------------------------------------|------------------------------------------------------------------|
| <code>DEPLOY_DIR</code>           | <code>/opt/rockinrio-backend</code>                       | Destino do <code>rsync</code> e raiz do compose no servidor      |
| <code>COMPOSE_PROJECT_NAME</code> | <code>rockinrio-backend</code>                            | Namespace do projeto Docker Compose                              |
| <code>IMAGE_CI</code>             | <code>rockinrio-backend:ci-\${CI_COMMIT_SHORT_SHA}</code> | Tag da imagem por commit (build/cleanup)                         |
| <code>IT_MYSQL_PORT</code>        | <code>13306</code>                                        | Porta de host do MySQL efêmero de teste (3306 já usada pela VPS) |

### 4.2 O arquivo `.env` no servidor (runtime)

O `.env` vive apenas em `/opt/rockinrio-backend/.env`, fora do git e preservado pelo `rsync` (`--exclude .env`). É a fonte de todos os segredos de runtime consumidos pelo `docker-compose.yml` via `env_file`. Base em `.env.example`:

| Grupo                   | Chaves                                                                     | Notas críticas                                                                               |
|-------------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Banco                   | DB_HOST, DB_PORT, DB_NAME, DB_URL, DB_USERNAME, DB_PASSWORD                | No compose, DB_HOST é sobrescrito para host.docker.internal                                  |
| Segredos (obrigatórios) | JWT_SECRET, RESOURCE_ID_SECRET                                             | Mín. 32 chars. A app <b>NÃO</b> sobe sem eles (fail-fast). Gerar com openssl rand -base64 48 |
| JWT                     | JWT_EXPIRATION_MS, JWT_REFRESH_EXPIRATION_MS                               | Access 24h / refresh 30d por padrão                                                          |
| Swagger                 | SPRINGDOC_ENABLED                                                          | false em produção                                                                            |
| Rate limit              | LOGIN_RATE_LIMIT_*, PUBLIC_RATE_LIMIT_*                                    | Anti força-bruta (login) e anti-abuso (portal público)                                       |
| CORS                    | CORS_ALLOWED_ORIGINS, CORS_ALLOWED_ORIGIN_PATTERNS, CORS_ALLOW_CREDENTIALS | Em prod: domínios REAIS e esvaziar patterns de localhost                                     |
| Arquivos                | APP_ARQUIVOS_DIR, MULTIPART_MAX_REQUEST_SIZE                               | Dir montado como volume ./arquivos                                                           |
| S3 (opcional)           | APP_S3_*, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY                         | Credenciais via env ou IAM role; nunca no banco                                              |
| Runtime                 | HOST_PORT                                                                  | Porta publicada pelo container (dev 8080 · prod 8203)                                        |

**Fail-fast de segredos.** Em application.properties, app.jwt.secret e app.resource-id.secret não têm valor default. Se o .env não os fornecer, a aplicação não inicia — o container entra em loop de restart. Sempre validar o .env antes de subir.

### 4.3 Como o segredo NÃO vaza no CI

- O .env é excluído do rsync e do contexto de build (.dockerignore) — nunca entra na imagem.
- Gitleaks escaneia o histórico e a árvore; .gitleaks.toml mantém allowlist só para placeholders de exemplo.
- Credenciais de produção devem viver como CI/CD variables mascaradas/protegidas no GitLab, ou diretamente no .env do servidor.

## 5. Anatomia do pipeline — .gitlab-ci.yml

### 5.1 Estágios e ordem de execução

```
stages:
  - build      # 1. Constrói a imagem e extrai o fat-jar
  - test       # 2. Gate de testes com MySQL efêmero
  - security   # 3. SCA + SAST + secret scanning (paralelo)
  - deploy     # 4. rsync + docker compose up (dev/prod)
```

### 5.2 Regras de branch (quando o pipeline roda)

Uma âncora YAML reutilizável restringe a execução às branches main e develop. Qualquer outra branch (feature, hotfix) não dispara build/test/security.

```
.branch_main_develop: &branch_main_develop
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
    - if: $CI_COMMIT_BRANCH == "develop"
```

### 5.3 Grafo de dependências (needs)

| Job                                                | needs | Roda em       |
|----------------------------------------------------|-------|---------------|
| build                                              | —     | main, develop |
| test                                               | build | main, develop |
| sca_dependencias · sast_semgrep · secret_detection | build | main, develop |
| deploy_development                                 | test  | develop       |
| deploy_production                                  | test  | main          |

**Observação importante.** O deploy depende apenas de test (não dos jobs de segurança). Como os jobs de segurança são allow\_failure, o deploy prossegue mesmo com findings — até a equipe promovê-los a gate bloqueante (§8.4).

## 6. Stage BUILD — imagem e extração do fat-jar

```
build:
  stage: build
  tags: [deploy]
  <<: *branch_main_develop
  script:
    - docker build -t "${IMAGE_CI}" .
    # Extrai o fat-jar para o SCA (Trivy) escanear offline
    - docker rm -f "ci-extract-${CI_JOB_ID}" >/dev/null 2>&1 || true
    - docker create --name "ci-extract-${CI_JOB_ID}" "${IMAGE_CI}"
    - docker cp "ci-extract-${CI_JOB_ID}:/app/app.jar" ./app.jar
    - docker rm -f "ci-extract-${CI_JOB_ID}" >/dev/null 2>&1 || true
  artifacts:
    paths: [ app.jar ]
    expire_in: 1 hour
```

1. docker build executa o Dockerfile até o estágio final (runtime), produzindo rockinrio-backend:ci-<sha>. Nesse processo o estágio build roda mvn -DskipTests package, gerando target/app.jar.
2. Um container temporário é criado (não iniciado) só para copiar o artefato de dentro dele.
3. docker cp extrai /app/app.jar para o workspace do CI.
4. O container temporário é removido; o app.jar vira artefato (validade 1h) consumido pelo SCA.

**Por que extrair o jar?** O Trivy escaneia o fat-jar offline — todas as dependências já estão embarcadas. Isso evita re-resolver o Maven Central (HTTP 429), tornando o SCA determinístico e rápido.

## 7. Stage TEST — MySQL efêmero e gate real

```
test:
  stage: test
  tags: [deploy]
  needs: [build]
  variables:
    IT_MYSQL_PORT: "13306" # 3306 já usada pelo banco da VPS
  script:
    - docker run -d --name "it-mysql-${CI_JOB_ID}" -e MYSQL_ALLOW_EMPTY_PASSWORD=1
      -e MYSQL_DATABASE=achados_perdidos_it -p ${IT_MYSQL_PORT}:3306 mysql:8.0
      --default-authentication-plugin=mysql_native_password
    - for i in $(seq 1 40); do docker exec ... mysqladmin ping ... && break; sleep 2; done
    - docker build --target test --network host
      --build-arg TEST_DB_URL="jdbc:mysql://127.0.0.1:${IT_MYSQL_PORT}/achados_perdidos_it?..."
      -t "rockinrio-backend-test:${CI_COMMIT_SHORT_SHA}" .
  after_script:
    - docker rm -f "it-mysql-${CI_JOB_ID}" >/dev/null 2>&1 || true
```

A base de testes (`IntegrationTestBase`) usa `assumeTrue(canConnect(), ...)` no `@BeforeAll`. Sem banco alcançável, os testes de integração são pulados silenciosamente — o gate ficaria verde sem validar nada. Por isso o CI sobe um `mysql:8.0` descartável na porta 13306, aguarda via `mysqladmin ping` (até  $40 \times 2s = 80s$ ), roda `docker build --target test` (dispara `mvn -B test`) com `--network host`, e remove o MySQL no `after_script`.

**Este é o gate bloqueante.** Se qualquer teste falhar, o `docker build --target test` retorna erro e o stage `test` falha, impedindo o deploy (que depende de `test`). É o principal portão de qualidade do pipeline.

## 8. Stage SECURITY — SCA, SAST e secret scanning

Três jobs rodam em paralelo, todos com `needs: [build]` e `allow_failure: true`. São uma "avaliação contínua de risco": ficam visíveis (vermelhos quando há achados) mas não bloqueiam o deploy até a triagem inicial.

### 8.1 SCA — Trivy (A06)

```
sca_dependencias:
  image: aquasec/trivy:latest
  cache: { key: trivy-db, paths: [ .trivycache/ ] }
  script:
    - trivy fs --scanners vuln --severity HIGH,CRITICAL --exit-code 1 --format table app.jar
  allow_failure: true
```

### 8.2 SAST — Semgrep (A05/A08)

```
sast_semgrep:
  image: semgrep/semgrep:latest
  script:
    - semgrep scan --error --config "p/java" --config "p/secrets" --config "p/owasp-top-ten"
      --exclude-rule dockerfile.security.missing-user.missing-user .
  allow_failure: true
```

A regra `missing-user` (container como root) é excluída por ora: o compose faz bind-mount de `./arquivos`, e rodar como não-root exigiria ajustar permissões do volume no servidor.



## 8.3 Secret scanning — Gitleaks

```
secret_detection:
  image: zricethezav/gitleaks:latest
  script:
    - gitleaks detect --source . --config .gitleaks.toml --redact --verbose --exit-code 1
  allow_failure: true
```

**8.4 Como promover um job a GATE bloqueante.** Após a primeira triagem, remova `allow_failure: true` do job. A partir daí, um HIGH/CRITICAL faz o pipeline falhar. Recomenda-se começar por `secret_detection` (tolerância zero) e depois SCA.

## 9. Stage DEPLOY — publicação em dev e produção

```
.deploy_template: &deploy_template
stage: deploy
before_script:
  - # instala rsync (apk/apt) se ausente
script:
  - mkdir -p "${DEPLOY_DIR}"
  - rsync -a --delete --no-owner --no-group
    --exclude .env --exclude .git --exclude target
    --exclude arquivos --exclude paineladmin --exclude portalpublico ./ "${DEPLOY_DIR}/"
  - cd "${DEPLOY_DIR}"
  - test -f .env || (echo "ERRO - crie .env..." && exit 1)
  - docker compose down --remove-orphans --timeout 30 || true
  - docker rm -f rockinrio-backend 2>/dev/null || true
  - docker compose up -d --build --remove-orphans --force-recreate
  - docker compose ps
  - IMAGE_CI="${IMAGE_CI}" sh scripts/ci-docker-cleanup.sh
```

O rsync sincroniza o repositório para `/opt/rockinrio-backend` (excluindo `.env`, uploads, build local e frontends), valida a presença do `.env`, para a stack atual, sobe a nova versão com `--build --force-recreate`, imprime `docker compose ps` e executa a limpeza.

### 9.2 Jobs concretos por ambiente

|                  | deploy_development    | deploy_production              |
|------------------|-----------------------|--------------------------------|
| Branch (rule)    | develop               | main                           |
| resource_group   | rockinrio-backend-dev | rockinrio-backend-prod         |
| Environment      | development           | production (Swagger desligado) |
| HOST_PORT típico | 8080                  | 8203                           |

**Estratégia de indisponibilidade.** O deploy usa `down → up --build`: há uma janela de downtime enquanto a imagem é reconstruída e o container reinicia. Planeje deploys em horários de baixo tráfego. Blue-green não está implementado neste pipeline. O **health check** (§10.3) reduz o risco ao permitir que orquestrador/operador só considerem o container pronto quando `healthy`.

## 10. Dockerfile multi-stage, runtime e health check

```
FROM maven:3.9-eclipse-temurin-25 AS base
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline          # camada cacheável de dependências
COPY src ./src

FROM base AS test                        # usado só no stage "test" do CI
ARG TEST_DB_URL                         # falha aqui = gate de teste vermelho
ENV TEST_DB_URL=${TEST_DB_URL} ...
RUN mvn -B test

FROM base AS build                      # gera target/app.jar
RUN mvn -B -DskipTests package

FROM eclipse-temurin:25-jre-alpine      # runtime enxuto (JRE only)
WORKDIR /app
ENV SERVER_PORT=8080
COPY --from=build /app/target/app.jar app.jar
RUN mkdir -p arquivos
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --start-period=45s --retries=3 \
  CMD wget -q -O - "http://127.0.0.1:${SERVER_PORT}/actuator/health" | grep -q '"status":"UP"' || exit
  1
CMD ["java", "-jar", "app.jar"]
```

O uso de JRE Alpine no estágio final reduz drasticamente o tamanho e a superfície de ataque (sem Maven, sem JDK, sem código-fonte). O `wget` usado no health check vem do busybox da própria imagem Alpine.

### 10.2 Runtime — `docker-compose.yml`

```
services:
  backend:
    build: .
    image: rockinrio-backend:latest
    container_name: rockinrio-backend
    restart: unless-stopped
    ports:
      - "${HOST_PORT:-8080}:8080"
    env_file: [ .env ]
    environment:
      SERVER_PORT: "8080"
      DB_HOST: host.docker.internal      # alcança o MySQL da VPS
      APP_ARQUIVOS_DIR: /app/arquivos
    extra_hosts:
      - "host.docker.internal:host-gateway"
    volumes:
      - ./arquivos:/app/arquivos        # uploads persistidos no host
    healthcheck:                        # rev. 1.1 — ver §10.3
      test: ["CMD", "wget", "-q", "-O", "-", "http://127.0.0.1:8080/actuator/health"]
      interval: 30s
      timeout: 5s
      start_period: 45s
      retries: 3
```

### 10.3 Health check rev. 1.1

O container expõe um endpoint de saúde dedicado, consumido pelo Docker/orquestrador e por eventual load balancer:

- **Endpoint:** `GET /actuator/health` (Spring Boot Actuator). Retorna `{"status":"UP"}` quando aplicação e dependências essenciais (incl. datasource) estão saudáveis; `503` caso contrário.
- **Exposição restrita:** apenas `health` é publicado (`management.endpoints.web.exposure.include=health`), **sem detalhes** (`show-details=never`) — nenhuma informação de diagnóstico vaza, alinhado ao relatório de segurança (M-04 / A05). Liberado sem autenticação no `SecurityConfig`; os demais endpoints de management não são expostos.
- **Definição:** `HEALTHCHECK` no Dockerfile (imagem auto-descritiva) e bloco `healthcheck` no compose (runtime da VPS). `start-period` de 45s cobre o boot da JVM/Spring; 3 tentativas de 30s antes de marcar `unhealthy`.
- **Efeito operacional:** `docker compose ps` passa a exibir `STATUS = healthy` e `docker inspect ... {{.State.Health.Status}}` (§15.1) retorna o estado real — antes retornava vazio por não haver health check definido.

Validação manual: `curl -fsS http://127.0.0.1:8203/actuator/health` na VPS deve retornar `{"status":"UP"}` com HTTP 200.

## 11. Nginx, Cloudflare e exposição da API

Referência em `deploy/nginx/api-rockinrio.xpto.com.br.conf`. Instalar em `/etc/nginx/sites-available/` com symlink em `sites-enabled/`.

```
server {
    listen 80;
    server_name api-rockinrio.xpto.com.br;
    location / {
        proxy_pass http://127.0.0.1:8203;           # HOST_PORT do container (prod)
        proxy_http_version 1.1;
        proxy_set_header Host                api-rockinrio.xpto.com.br; # hífen: Tomcat rejeita "-"
        proxy_set_header X-Forwarded-Host    api-rockinrio.xpto.com.br; # domínio real
        proxy_set_header X-Real-IP           $remote_addr;
        proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto  https;           # TLS terminado no Cloudflare
        proxy_read_timeout 60s;
        client_max_body_size 20m;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/api-rockinrio.conf /etc/nginx/sites-enabled/
sudo nginx -t           # valida a sintaxe
sudo systemctl reload nginx
```

**Consistência de portas e limites.** Alinhe `HOST_PORT` (.env), `proxy_pass` (Nginx) e a porta física livre na VPS. Além disso, `client_max_body_size` (20m) deve acompanhar `MULTIPART_MAX_REQUEST_SIZE` — senão uploads grandes tomam 413 no proxy antes de chegar à aplicação. A app confia em headers de proxy via `server.forward-headers-strategy=NATIVE` (RemotelpValve), refletindo o IP real sem aceitar spoofing (rate-limit e auditoria).

## 12. Banco de dados e migrations SQL

**As migrations NÃO são aplicadas pelo pipeline.** A aplicação roda com `spring.jpa.hibernate.ddl-auto=validate` em produção — o Hibernate valida que o schema bate com as entidades, mas não cria nem altera tabelas. Os scripts `scripts/0XX_*.sql` devem ser aplicados manualmente ao MySQL, em ordem numérica, antes de subir a versão que os requer. Se o schema divergir, a aplicação não inicia.

Os scripts vão de `018_Fluxo_Processos_Documento.sql` a `056_Auditoria_Snapshots_Completos.sql` (numeração incremental). Contêm DDL (tabelas, índices, FKs), seeds e correções. Aplicar na ordem do número, cada um uma única vez.

1. Backup antes de qualquer alteração: `mysqldump achados_perdidos > backup_$(date +%F).sql`
2. Identifique o último script já aplicado (mantenha um registro/tabela de controle).
3. Aplique os novos em ordem: `mysql achados_perdidos < scripts/0XX_*.sql`
4. Só então faça o merge/push que dispara o deploy da nova versão.

**Regra de ouro: migration primeiro, deploy depois.** Como o deploy é automático no push para `main / develop`, aplique o SQL no banco antes do merge. Caso contrário, o novo container falha na validação de schema e entra em loop de restart.

## 13. Procedimento de bootstrap (primeiro deploy)

Execute uma única vez ao preparar um ambiente novo (dev ou prod). Depois, os deploys são automáticos via push.

1. Provisionar a VPS: Docker Engine 24+, Compose v2, MySQL 8 acessível, Nginx.
2. Registrar o GitLab Runner com a tag `deploy`, com acesso ao Docker daemon e ao diretório de deploy.
3. Criar o diretório: `sudo mkdir -p /opt/rockinrio-backend`
4. Criar o banco (`CREATE DATABASE achados_perdidos;`) e aplicar as migrations 018...056 em ordem (§12).
5. Criar o `.env` em `/opt/rockinrio-backend/.env` a partir do `.env.example`, com valores REAIS:

```
JWT_SECRET=$(openssl rand -base64 48)
RESOURCE_ID_SECRET=$(openssl rand -base64 48)
DB_PASSWORD=<senha real do MySQL>
SPRINGDOC_ENABLED=false           # produção
CORS_ALLOWED_ORIGINS=https://painel_rockinrio...,https://app_rockinrio...
CORS_ALLOWED_ORIGIN_PATTERNS=    # vazio em produção
HOST_PORT=8203                    # produção
```

6. Definir a senha do admin (BCrypt) no banco: `UPDATE usuario SET NM_Senha='$2a$10$...' WHERE NM_Login='admin';`
7. Configurar o Nginx (§11) e recarregar.
8. Disparar o primeiro deploy: push/merge para `develop` (dev) ou `main` (prod).
9. Validar (§15): `docker compose ps` (esperado healthy), logs, e `curl` ao `/actuator/health` através do Nginx.

**Checklist de sanidade pré-primeiro-deploy.** `.env` existe com JWT/RESOURCE secrets ≥32 chars · banco criado e migrations aplicadas · MySQL alcançável via `host.docker.internal` · porta `HOST_PORT` livre · Nginx apontando para a porta certa · runner com tag `deploy` online.

# 14. Deploy contínuo, rollback e limpeza

## 14.1 Deploy contínuo (dia a dia)

- 1. (Se houver mudança de schema) aplicar migrations no banco — §12.
- 2. `git push` para `develop` → deploy automático em Development.
- 3. Após validação em dev, merge para `main` → deploy automático em Production.
- 4. Acompanhar o job de deploy no GitLab e a saída de `docker compose ps` (status `healthy`).

## 14.2 Rollback

O pipeline não tem botão de rollback nativo; use uma das estratégias:

| Estratégia                | Como                                                                                    | Quando                                                          |
|---------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Revert de commit          | <code>git revert &lt;sha&gt;</code> + push → redeploy da versão anterior                | Regressão de código; forma mais limpa                           |
| Re-run de pipeline antigo | No GitLab, re-executar o pipeline de um commit estável anterior                         | Quando o código anterior já passou pelo gate                    |
| Imagem CI anterior        | <code>docker tag rockinrio-backend:ci-&lt;sha_antigo&gt; ...</code> e subir manualmente | Emergência; a imagem ainda existe (cleanup mantém as 3 últimas) |

**Rollback e banco.** Se o release avançou o schema, o rollback de código pode falhar na validação contra o schema novo. Migrations idealmente são retrocompatíveis (aditivas). Reverter schema exige script de down manual e backup — planeje antes.

## 14.3 Limpeza automática pós-deploy

`scripts/ci-docker-cleanup.sh` roda ao fim de todo deploy: remove containers parados >24h e imagens dangling; mantém as `KEEP_CI_IMAGES` (default 3) imagens `ci-*` mais recentes; poda cache de build >168h e redes órfãs. **Nunca remove volumes nem imagens em uso** — uploads e banco estão seguros.

# 15. Observabilidade, health check e troubleshooting

## 15.1 Comandos de diagnóstico no servidor

```
cd /opt/rockinrio-backend
docker compose ps                # estado do container (agora com healthy/unhealthy)
docker compose logs -f --tail=200 backend    # logs em tempo real
docker inspect rockinrio-backend --format '{{.State.Health.Status}} {{.State.Status}}'
docker stats rockinrio-backend --no-stream   # CPU/memória
curl -fsS http://127.0.0.1:8203/actuator/health # health (prod) → {"status":"UP"}
```

## 15.2 Matriz de sintomas

| Sintoma                                  | Causa provável                                                                  | Ação                                                                                     |
|------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Container em loop de restart / unhealthy | <code>JWT_SECRET</code> / <code>RESOURCE_ID_SECRET</code> ausentes ou <32 chars | Corrigir <code>.env</code> , <code>docker compose up -d --force-recreate</code>          |
| unhealthy mas app "sobe"                 | MySQL inacessível → <code>/actuator/health</code> = DOWN (datasource)           | Verificar <code>extra_hosts</code> /MySQL; ver logs. É o health check cumprindo o papel. |

| Sintoma                         | Causa provável                                            | Ação                                                    |
|---------------------------------|-----------------------------------------------------------|---------------------------------------------------------|
| App sobe e cai — erro de schema | Migration não aplicada ( <code>ddl-auto=validate</code> ) | Aplicar SQL pendente (§12) e recriar                    |
| HTTP 400 no Nginx               | Header Host com underscore chegando ao Tomcat             | Conferir <code>proxy_set_header</code> Host com hífen   |
| HTTP 413 em upload              | <code>client_max_body_size</code> < tamanho do arquivo    | Alinhar Nginx e <code>MULTIPART_MAX_REQUEST_SIZE</code> |
| CORS bloqueado no frontend      | Origem não listada em <code>CORS_ALLOWED_ORIGINS</code>   | Adicionar domínio real; recriar container               |
| Deploy trava                    | <code>resource_group</code> serializando                  | Aguardar o pipeline anterior concluir                   |
| Stage test falha só às vezes    | MySQL efêmero não subiu no timeout (80s)                  | Re-executar; verificar carga do runner                  |

### 15.3 Onde olhar primeiro

1. Log do job GitLab — falha de build/test/deploy.
2. `docker compose logs` — falha de runtime (schema, secret, banco).
3. `/actuator/health` e `docker inspect ... Health` — prontidão do container.
4. `nginx -t` e log do Nginx — proxy/headers/TLS. MySQL — conectividade e schema.

## 16. Checklist operacional e hardening

### 16.1 Checklist antes de cada release em produção

- Migrations necessárias aplicadas ao banco de produção (em ordem).
- Backup do banco realizado.
- `SPRINGDOC_ENABLED=false` no `.env` de produção.
- `CORS_ALLOWED_ORIGINS` só com domínios reais; patterns de localhost vazios.
- Gate de teste verde no pipeline.
- Findings de segurança triados (Trivy/Semgrep/Gitleaks).
- Health check `healthy` após subir ( `docker compose ps` ).
- Janela de baixo tráfego (há downtime no `down/up` ).
- Plano de rollback definido (revert vs. imagem anterior).

### 16.2 Recomendações de hardening (evolução do pipeline)

| Área               | Recomendação                                                                                                                                                                                                                                                                  |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Health check       | <b>Implementado (rev. 1.1)</b> <code>/actuator/health</code> restrito + <code>HEALTHCHECK</code> no Dockerfile e <code>compose</code> — §10.3. Evolução: usar <code>--wait</code> no <code>compose up</code> para o deploy só concluir com o container <code>healthy</code> . |
| Gates de segurança | Promover <code>secret_detection</code> e depois <code>sca_dependencias</code> a bloqueantes (remover <code>allow_failure</code> ) — §8.4.                                                                                                                                     |
| Zero downtime      | Avaliar blue-green / <code>healthcheck</code> + <code>--wait</code> no <code>compose</code> antes de derrubar o antigo.                                                                                                                                                       |
| Migrations         | Adotar Flyway/Liquibase para versionar e aplicar schema automaticamente no start.                                                                                                                                                                                             |
| Imagem non-root    | Resolver permissão do volume <code>./arquivos</code> e reabilitar a regra <code>missing-user</code> do Semgrep.                                                                                                                                                               |

| Área            | Recomendação                                                                                                                                                  |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Segredos        | Migrar do <code>.env</code> em disco para um cofre (Vault / GitLab masked+protected vars).                                                                    |
| Observabilidade | Centralizar logs (Loki/ELK) e métricas (Prometheus). Actuator já habilitado pode expor <code>/actuator/prometheus</code> de forma restrita quando necessário. |
| Pin de imagens  | Fixar digests de <code>trivy:latest</code> / <code>semgrep:latest</code> / <code>gitleaks:latest</code> para builds reprodutíveis.                            |

Documento gerado a partir da análise direta do repositório `api-achados-perdidos` (`.gitlab-ci.yml`, `Dockerfile`, `docker-compose.yml`, `application.properties`, `deploy/nginx/*.conf`, `scripts/*`). Mantenha este runbook versionado junto ao código e atualize-o a cada mudança no pipeline. — XPTO · Rock in Rio Backend · v1.1 · 24/07/2026.